Estrutura de Dados Lineares

Tabela Hash

Introdução

- Métodos de busca comparam chaves.
- Para algoritmos eficientes de busca, os elementos costumam estar ordenados.
- Ordenar os dados tem custo **O(n log n)**.
- Mesmo com os dados ordenados, a **busca binária** tem custo **O(log n)**.
- Busca ideal: acesso direto ao elemento, sem comparação de chaves: **O(1)**.

Introdução

- Um **array** permite acesso a um elemento conhecido por seu **índice** em tempo **O(1)**.
- No entanto, o array não possui mecanismo para localizar um item **a partir de sua chave**.
- As **tabelas Hash** generalizam essa ideia: calculam o índice a partir da chave, utilizando uma **função de hashing** para espalhar os elementos no vetor.
- Assim, cada **chave** é associada a uma **posição (índice)** dentro da tabela.

Introdução

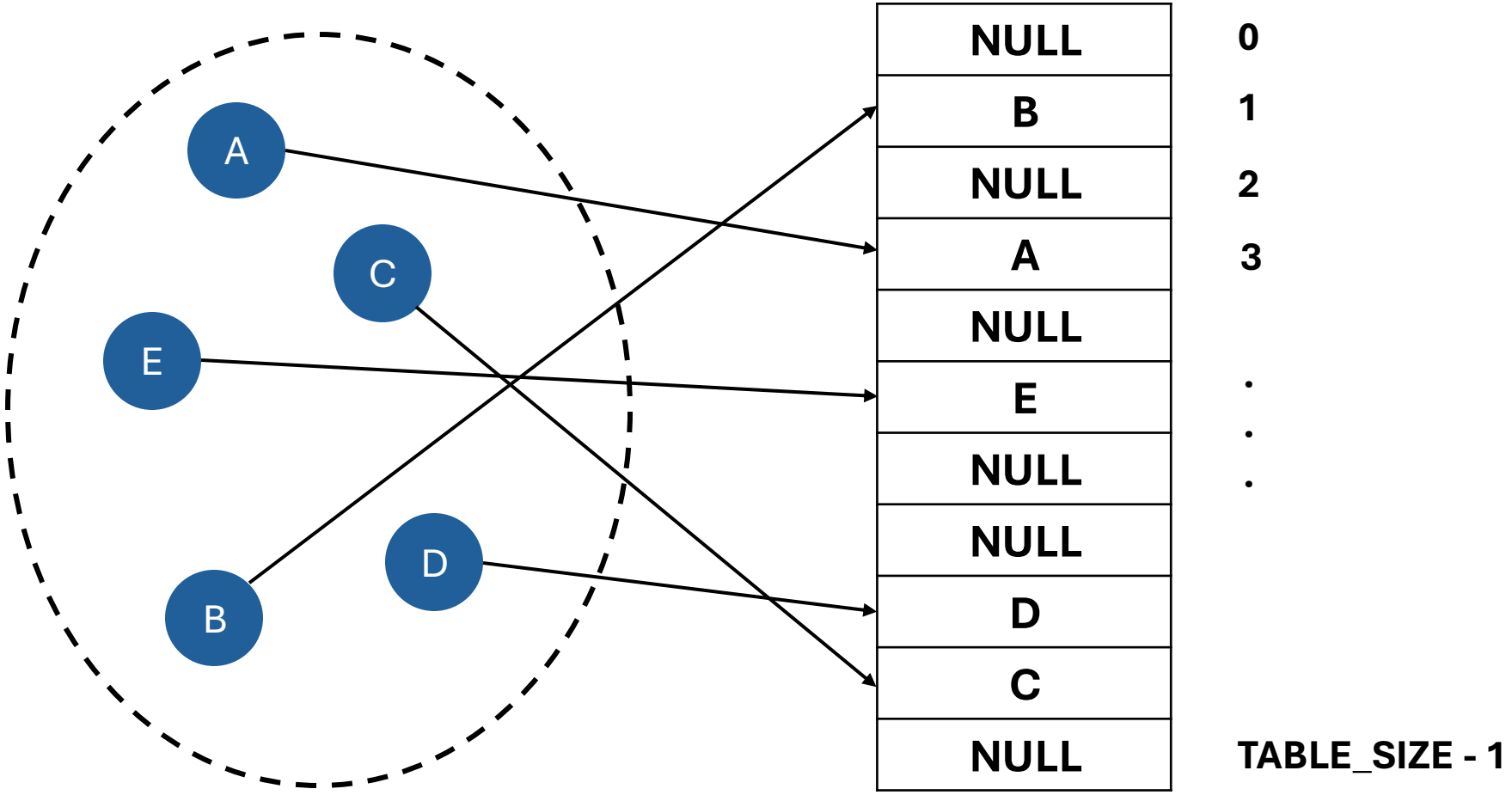


Figura 10.1 Exemplo de função de hashing mapeando as chaves A–E em posições distintas do array.

Introdução

- A função de *hashing* transforma uma **chave** em uma **posição (índice)** no vetor.
- Representação geral:

h(chave) = posição

h(‘A’) = 3

h(‘B’) = 1

h(‘C’) = 7

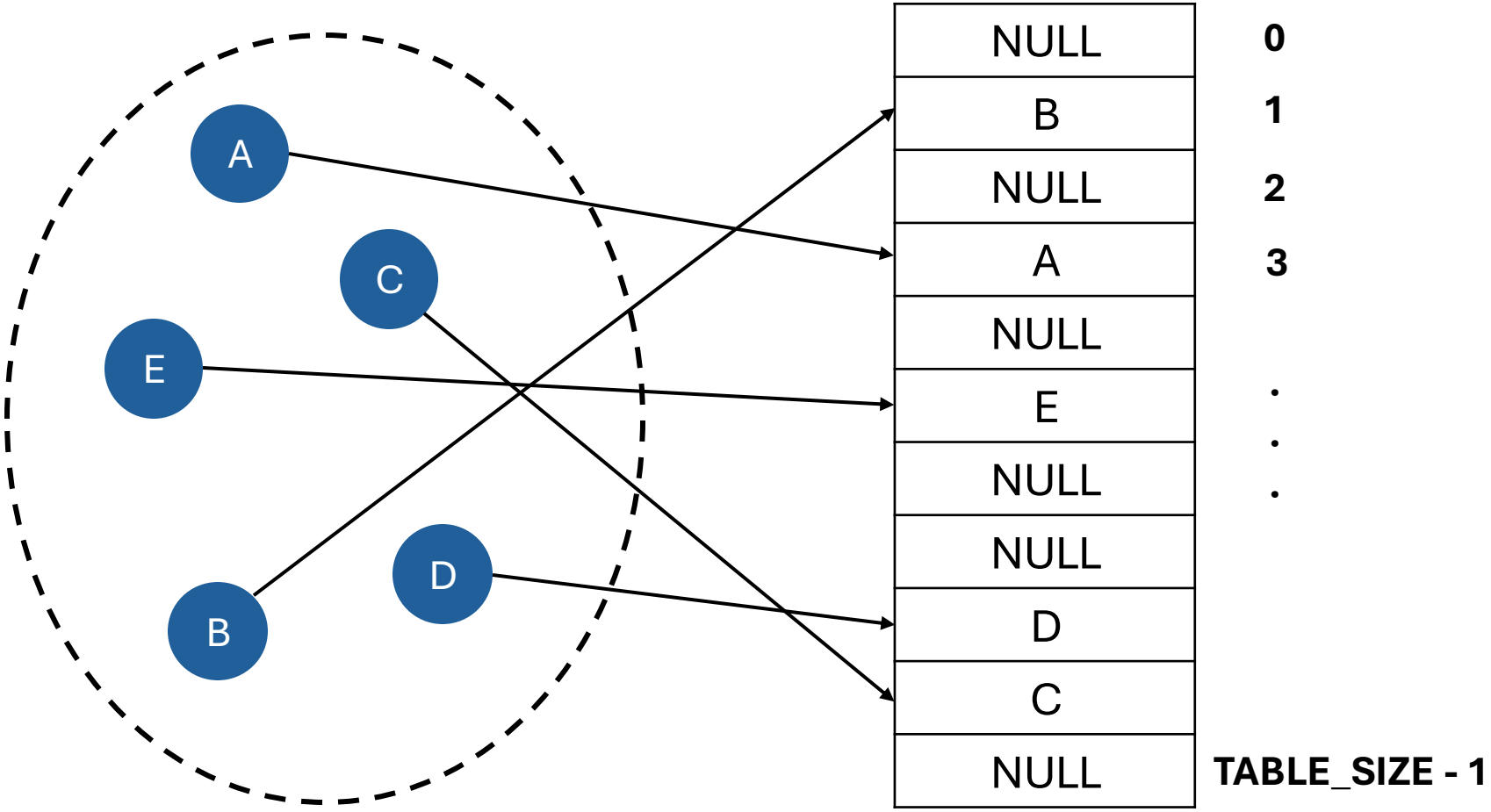


Figura 10.1 Exemplo de função de hashing mapeando as chaves A–E em posições distintas do array.

Introdução

- O objetivo da **função de hashing** é **espalhar uniformemente** as chaves no vetor.
- Essa distribuição reduz a probabilidade de **colisões** (duas chaves ocupando a mesma posição).

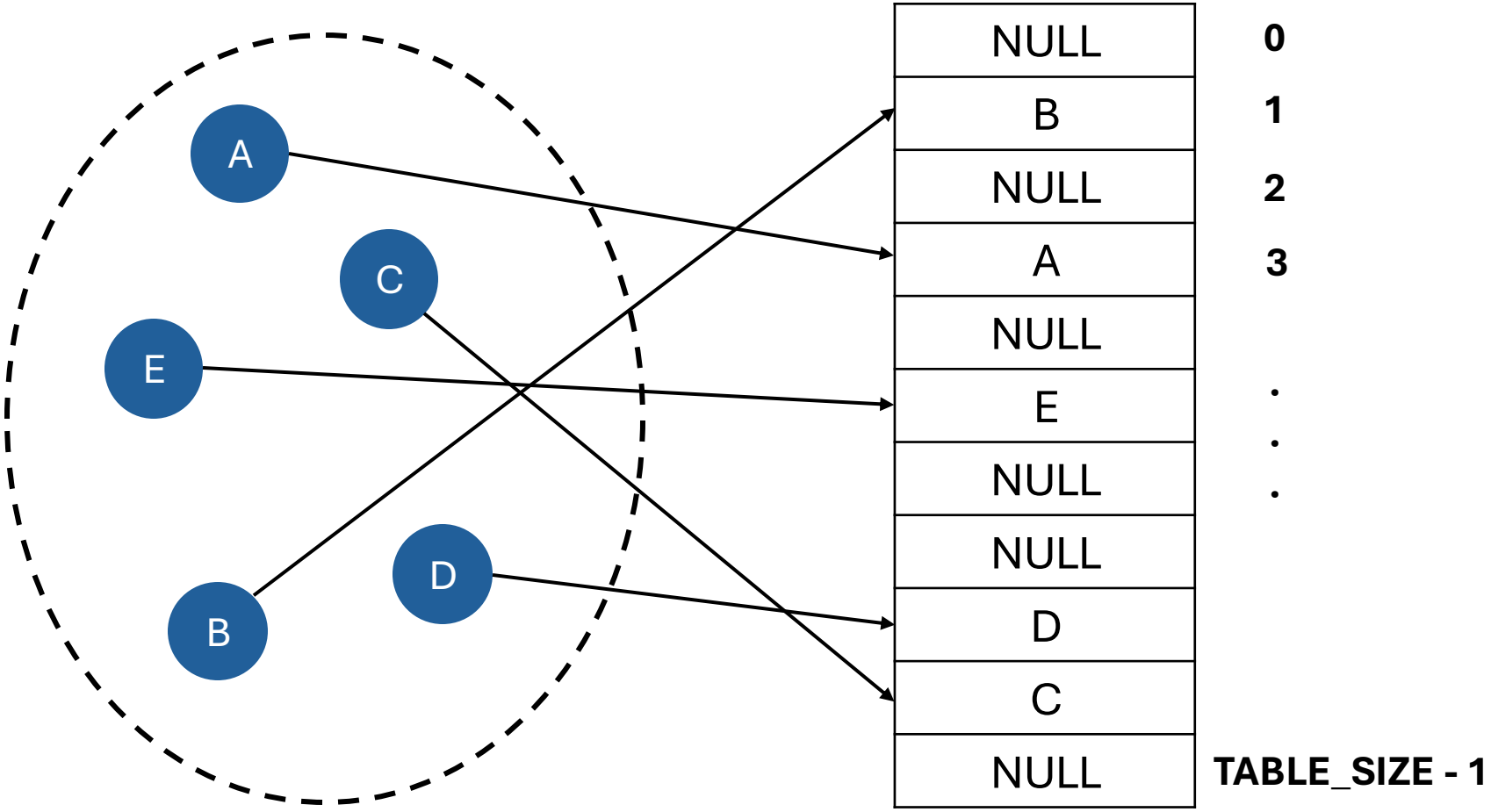


Figura 10.1 Exemplo de função de hashing mapeando as chaves A–E em posições distintas do array.

Introdução

- A tabela *Hash* permite acesso rápido aos dados
  - caso médio: **O(1)**
- No entanto, **duas chaves diferentes podem gerar o mesmo índice: *colisão***.
- **Colisões** não são erros, mas reduzem o desempenho da busca.
- O pior caso ocorre quando muitos elementos ocupam a mesma posição: custo **O(n)**.
- **Colisão:** duas (ou mais) chaves mapeadas para a mesma posição da tabela.

Aplicações

Estruturas de dados de acesso rápido

- Tabelas Hash são a base de **estruturas de dados de acesso rápido** usadas em várias linguagens modernas.
- Permitem **armazenar pares chave–valor** com busca média em **O(1)**.
- São amplamente usadas em:
  - dict (Python)
  - HashMap (Java)
  - unordered_map (C++)

alunos["Maria"] = 9.5 // insere

nota = alunos["Maria"] // busca O(1)

- A ideia é a mesma da tabela hash:
  - **a chave é convertida em um índice** por meio de uma **função de hashing**.

Aplicações

Senhas e autenticação

- Quando um usuário cria uma senha, o sistema **não armazena o texto original**.
- A senha passa por uma **função de hashing criptográfica**, gerando um valor único (*hash*).
- Durante o login, o sistema aplica a mesma função à senha digitada e **compara os hashes**.
- Mesmo pequenas alterações na senha produzem hashes totalmente diferentes.

Aplicações

Senhas e autenticação

- Exemplo simplificado:

Senha	Hash (SHA-256)
senha123	ef92b7…c1b2
senha124	b0e27f…8a4c

- Essa mesma ideia de transformar uma informação em um valor fixo também é usada em **funções de hashing criptográficas**, mas com o objetivo de **verificação e segurança**, não de armazenamento em tabelas.

Aplicações

- Estruturas de dados de alta performance (como dict em Python, HashMap em Java).
- Indexação em bancos de dados e mecanismos de busca.
- Verificação de integridade de arquivos (MD5, SHA).
- Tabela de símbolos em compiladores (associar identificadores e endereços).

Função de Hashing

- O objetivo é transformar uma **chave** (inteiro, string, matrícula, etc.) em um **índice do array** da tabela Hash.
- Essa transformação deve **espalhar uniformemente** as chaves nas posições.
- Um bom espalhamento: **menos colisões**, **busca mais rápida** (*O(1)* em média).
- Existem várias funções possíveis:
  - método da divisão
  - método da multiplicação
  - método da dobra

Função de Hashing

Método da divisão

- A função de *hashing* calcula a **posição no array** a partir da chave ***k***.
- O método da divisão é o mais simples e direto:

$$h(k) = k \bmod TABLE_SIZE$$

- O operador módulo retorna o **resto da divisão inteira da chave pelo tamanho da tabela**.
- Exemplo:

$$h(37) = 37 \bmod 11 = 4$$

- o elemento de chave 37 será armazenado na **posição** 4.

Função de Hashing

Método da divisão

- O tamanho da tabela (**TABLE_SIZE**) influencia diretamente a distribuição.
- Para reduzir colisões, recomenda-se:
  - escolher **números primos** como tamanho da tabela;
    - quebram padrões aritméticos das chaves (espalham melhor);
  - evitar potências de 2 (2, 4, 8, 16, ...).
    - geram padrões binários repetitivos (pior espalhamento).
- Exemplo:
  - TABLE_SIZE = 11 (primo)
  - melhor espalhamento das chaves e menor chance de padrões repetitivos.
  - evita que múltiplos de 10, 20, 30... caiam sempre no mesmo índice.

Função de Hashing

Método da multiplicação

- Usa uma **constante fracionária irracional A**, com valor entre 0 e 1.
- Calcula a posição com base na **parte fracionária do produto k × A**.

h(k) = parte_inteira(TABLE_SIZE × parte_fracionaria(k × A))

- A parte fracionária se comporta como um valor **"quase aleatório"**, o que garante bom espalhamento.
- Valor comumente usado:

  A = 0.6180339887 (inverso do número áureo)

Função de Hashing

Método da multiplicação

- Como A é **irracional**, a sequência de valores (k × A) nunca se repete.
- As partes fracionárias de (k × A) ficam **uniformemente distribuídas** entre 0 e 1.
- Isso **espalha as chaves** de forma mais equilibrada e **reduz colisões**.
- Funciona bem mesmo se **TABLE_SIZE não for primo**.
- Garante bom espalhamento mesmo quando as chaves **k** são valores consecutivos.

Função de Hashing

Tratando uma string como chave

- Sempre que desejamos inserir ou buscar um elemento na tabela Hash, usamos **parte do elemento** como **chave** para calcular a posição na tabela.
- No caso dos elementos serem constituídos unicamente de *strings*, podemos **calcular um valor numérico** a partir da *string*.
- Esse valor pode ser obtido **somando os valores ASCII dos caracteres** segundo uma regra que **leva em conta a ordem** (acumulação polinomial).
- **O resultado dessa função** pode então ser **utilizado como parâmetro** para uma **função de hashing** (divisão, multiplicação, etc.).

Função de Hashing

Tratando uma string como chave

- A função que transforma a *string* em inteiro devolve um **valor de chave**.
- A posição na tabela é então **calculada pela função de hashing** escolhida para a tabela.
- Exemplo com **método da divisão**:

h(str) = valorString(str) % TABLE_SIZE = pos;

- Assim, a partir da *string*, obtemos um índice do *array* com **custo O(1) na média**.

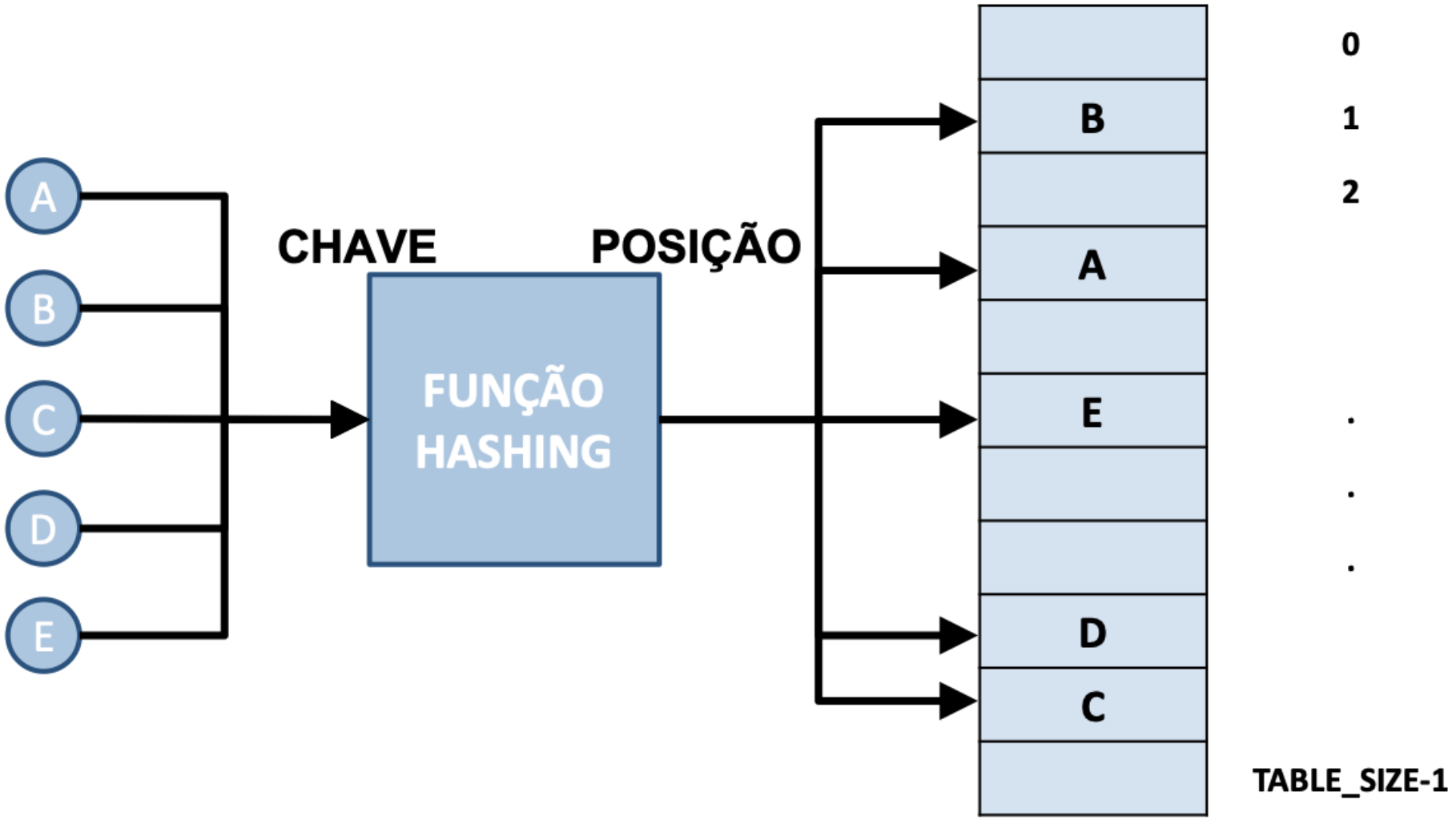
Função de Hashing

Tratando uma string como chave

```c
int valorString(char *str){
    int i, valor = 7;
    int tam = strlen(str);
    for(i=0; i < tam; i++)
        valor = 31 * valor + (int) str[i];
    return valor;
}
```

- Por que não simplesmente somar ASCII? Palavras com letras trocadas produzem o mesmo valor e geram colisão:
  - cama e maca resultam no mesmo somatório.

Função de Hashing



Hashing perfeito e imperfeito

- O desempenho de uma tabela *Hash* depende da função de *hashing* e do tamanho da tabela.
- Podemos classificar uma função de *hashing* como **imperfeita** ou **perfeita**.

Hashing imperfeito

- Uma função de *hashing* é dita imperfeita se, para duas chaves diferentes, a saída da função é a mesma posição na tabela.
- Podem ocorrer **colisões** das chaves armazenadas.
- A colisão **não é um erro**, mas diminui o desempenho.
- Normalmente tratamos colisões com **estruturas auxiliares** ou **estratégias de reposicionamento**.

Hashing perfeito

- Uma função de *hashing* é dita perfeita se nunca ocorre colisão.
- Chaves diferentes geram posições **diferentes** na tabela.
- Inserção e busca são sempre executadas em **O(1)**.
- Usado apenas quando **as chaves são conhecidas previamente**, como:
  - palavras reservadas de uma linguagem de programação;
  - pequenas tabelas estáticas.

Colisão

Causas comuns de colisão

- Tamanho da tabela pequeno em relação ao número de chaves.
- Função de *hashing* mal distribuída.
- Padrões nos dados (ex: várias chaves com prefixos semelhantes).
- Mesmo uma função uniforme terá colisões, elas são **teoricamente inevitáveis**.

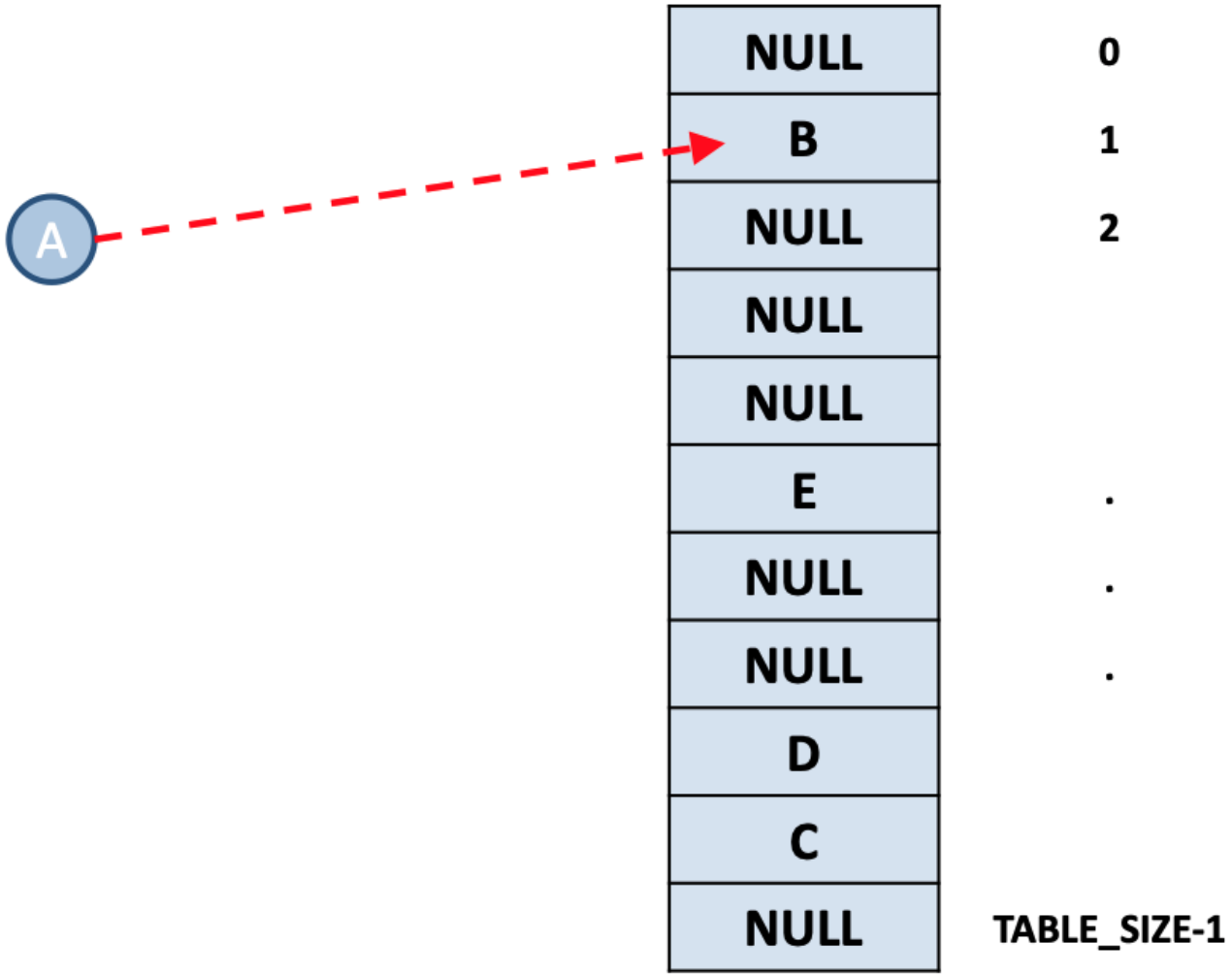
Colisão

Técnicas mais comuns para tratar coliões

- Existem duas abordagens principais:
- **Endereçamento aberto** (*open addressing*)
  - todos os elementos ficam **na própria tabela**.
  - quando há colisão, procura-se outra posição **vazia**.
- **Encadeamento separado** (*separate chaining*)
  - cada posição da tabela guarda o **início de uma lista** com os elementos que colidiram.

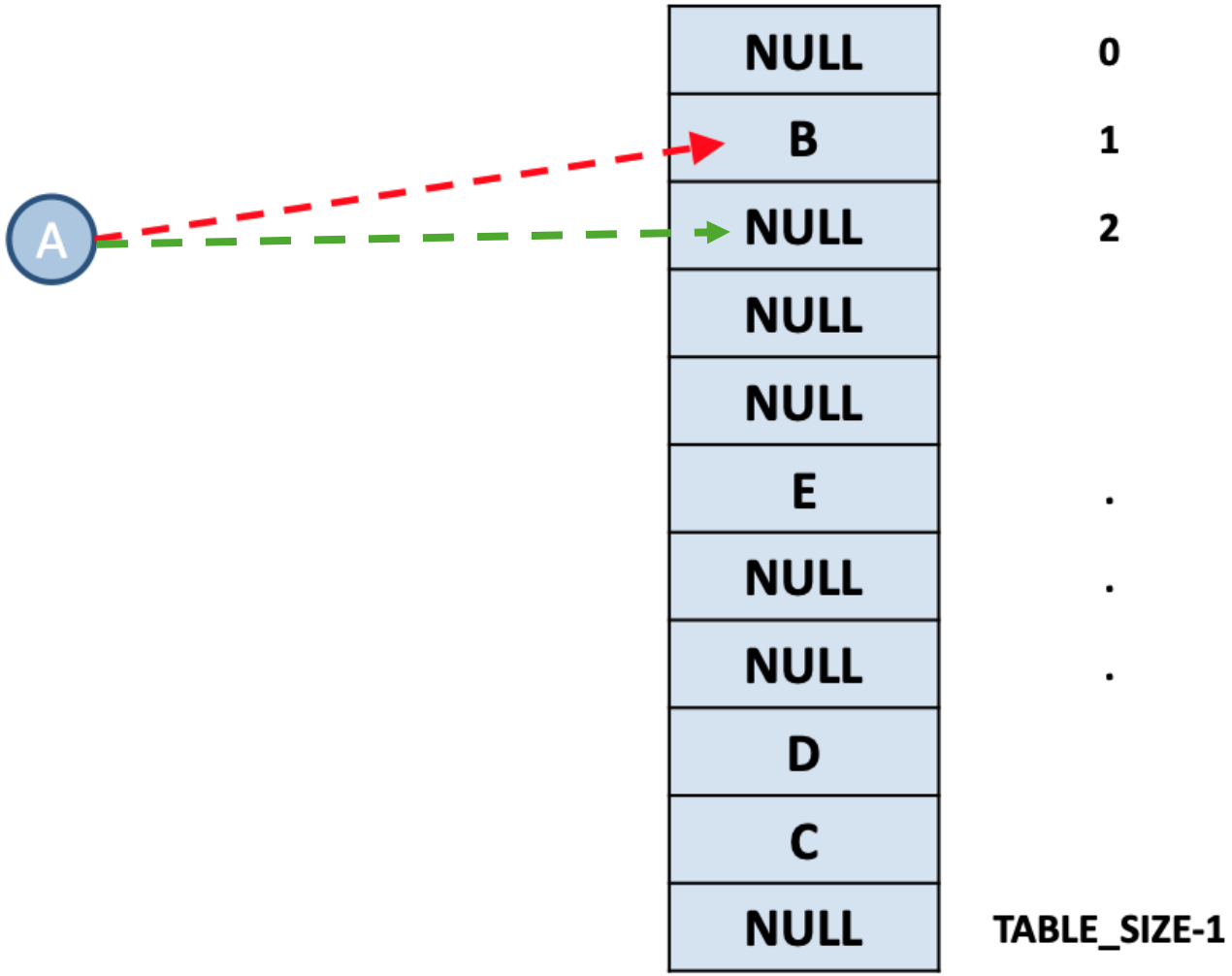
Endereçamento aberto

- Todos os elementos são armazenados na própria tabela *Hash*. Quando ocorre colisão, procura-se outra posição vazia dentro do array.



Endereçamento aberto

- Todos os elementos são armazenados na própria tabela *Hash*. Quando ocorre colisão, procura-se outra posição vazia dentro do array.



Endereçamento aberto

- Dispensa o uso de listas encadeadas.
- Percorre (ou “sonda”) a tabela até encontrar um espaço livre.
- Exige **mais cálculos de posição** em caso de colisão.
- No pior caso, o custo pode chegar a *O(n)*.

Endereçamento aberto

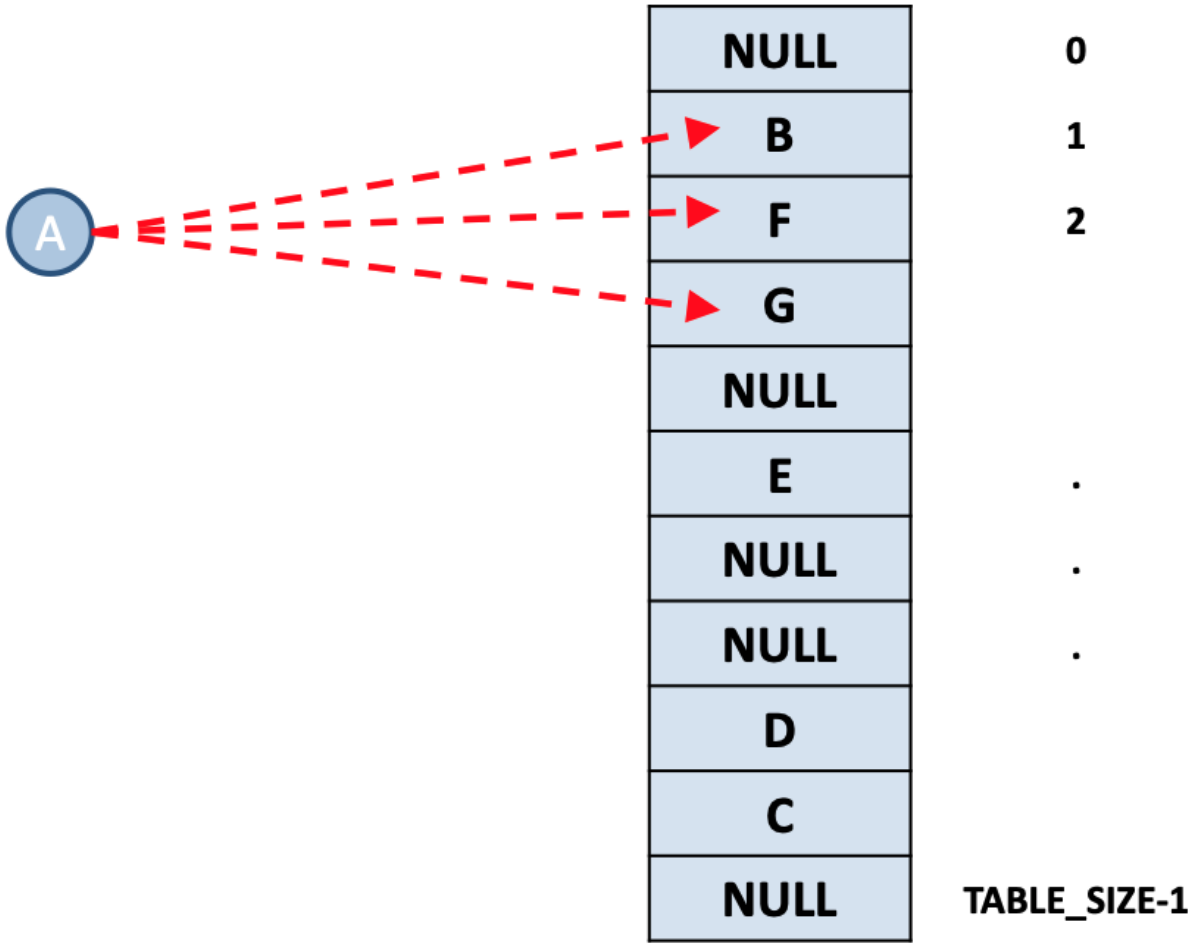
Vantagens

- Aproveita melhor a memória (sem ponteiros adicionais).
- Mantém todos os dados em uma única estrutura.
- Pode ser mais rápido para leituras sequenciais.

Endereçamento aberto

Desvantagens

- Cálculo repetido de posições quando há colisões.
- Dificuldade para **remoção** de elementos (precisa manter marcações).
- Maior risco de **agrupamentos** (posições consecutivas ocupadas).



Endereçamento aberto: Sondagem linear

- No caso de colisão, tenta-se a próxima posição disponível na sequência.

```
int sondagemLinear(int pos, int i, int TABLE_SIZE){
    return (pos + i) % TABLE_SIZE;
}
```

- Simples e rápida de implementar.
- Pode gerar **agrupamento primário,** blocos contíguos de posições ocupadas.

Endereçamento aberto: Sondagem linear

Inserção

Quando ocorre uma colisão:

- Calcula a posição inicial:
  - pos = funcao_hash(chave)

A

NULL	0
B	1
F	2
G	
NULL	
E	.
NULL	.
NULL	.
D	
C	
NULL	TABLE_SIZE-1

Endereçamento aberto: Sondagem linear

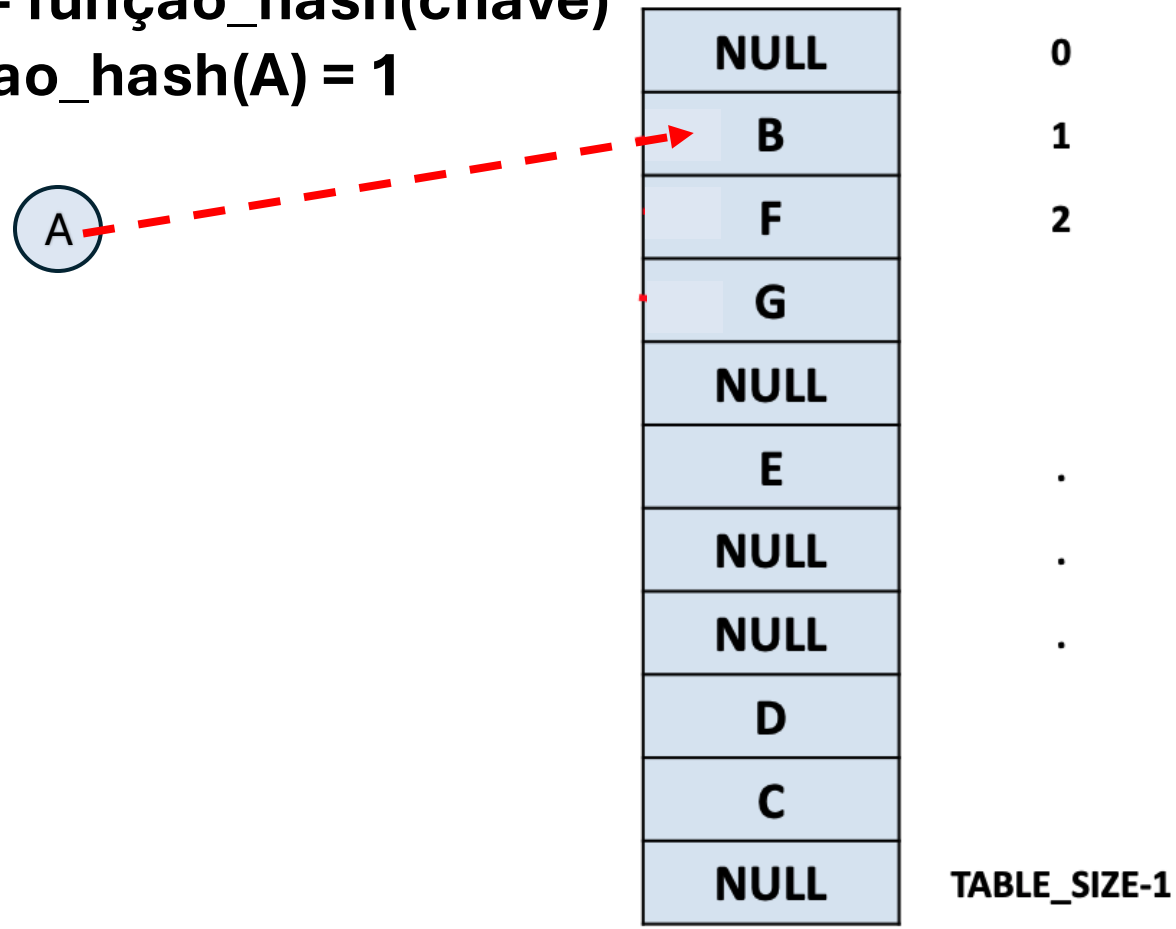
Inserção

Quando ocorre uma colisão:

- Calcula a posição inicial:
  - pos = funcao_hash(chave)
- Verifica se a posição está livre:
  - Se **vazia** → insere.
- Se **ocupada** → tenta a próxima.
  - Calcula nova posição:

    pos = (pos + 1) % TABLE_SIZE

pos = função_hash(chave)
funcao_hash(A) = 1



Endereçamento aberto: Sondagem linear

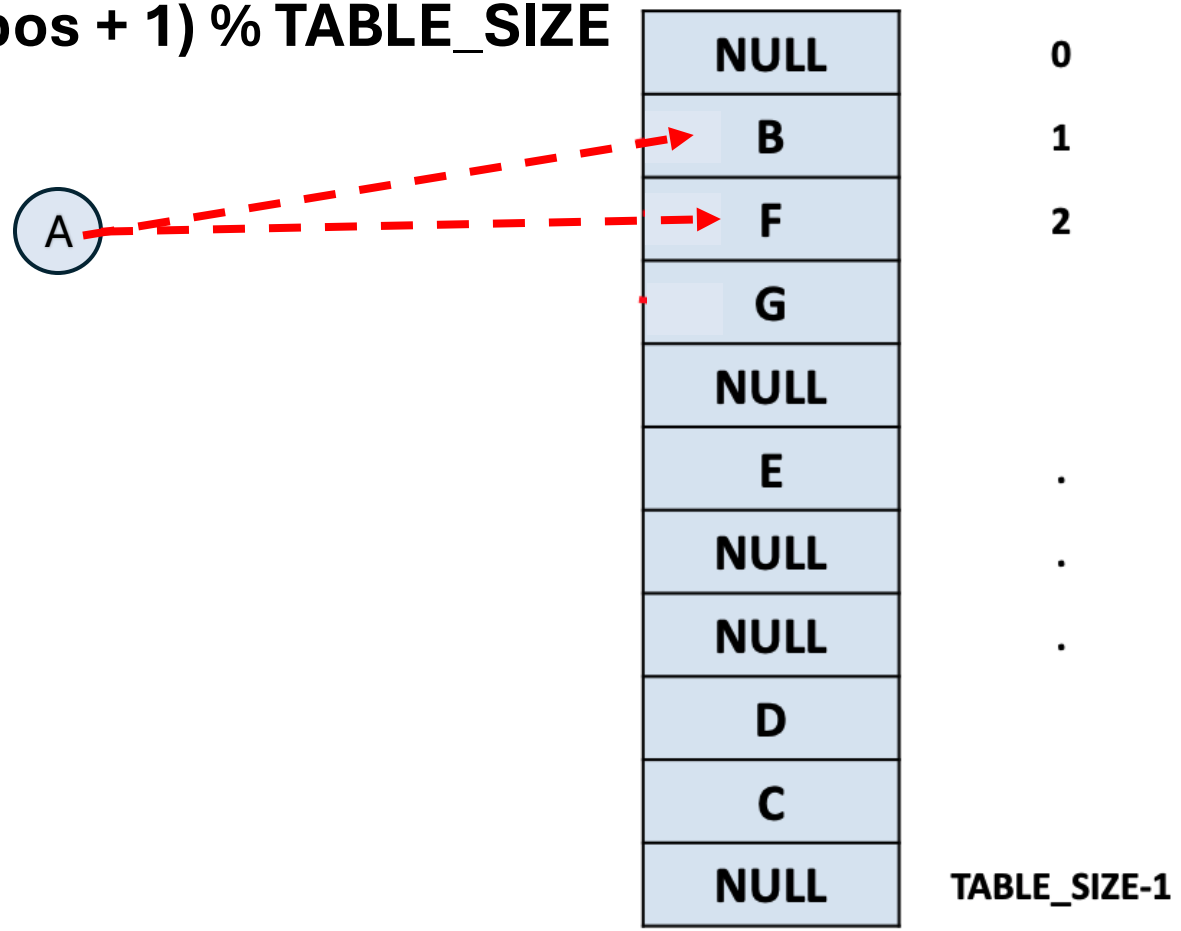
Inserção

Quando ocorre uma colisão:

- Calcula a posição inicial:
  - pos = funcao_hash(chave)
- Verifica se a posição está livre:
  - Se **vazia** → insere.
- Se **ocupada** → tenta a próxima.
  - Calcula nova posição:

    pos = (pos + 1) % TABLE_SIZE

pos = (pos + 1) % TABLE_SIZE



Endereçamento aberto: Sondagem linear

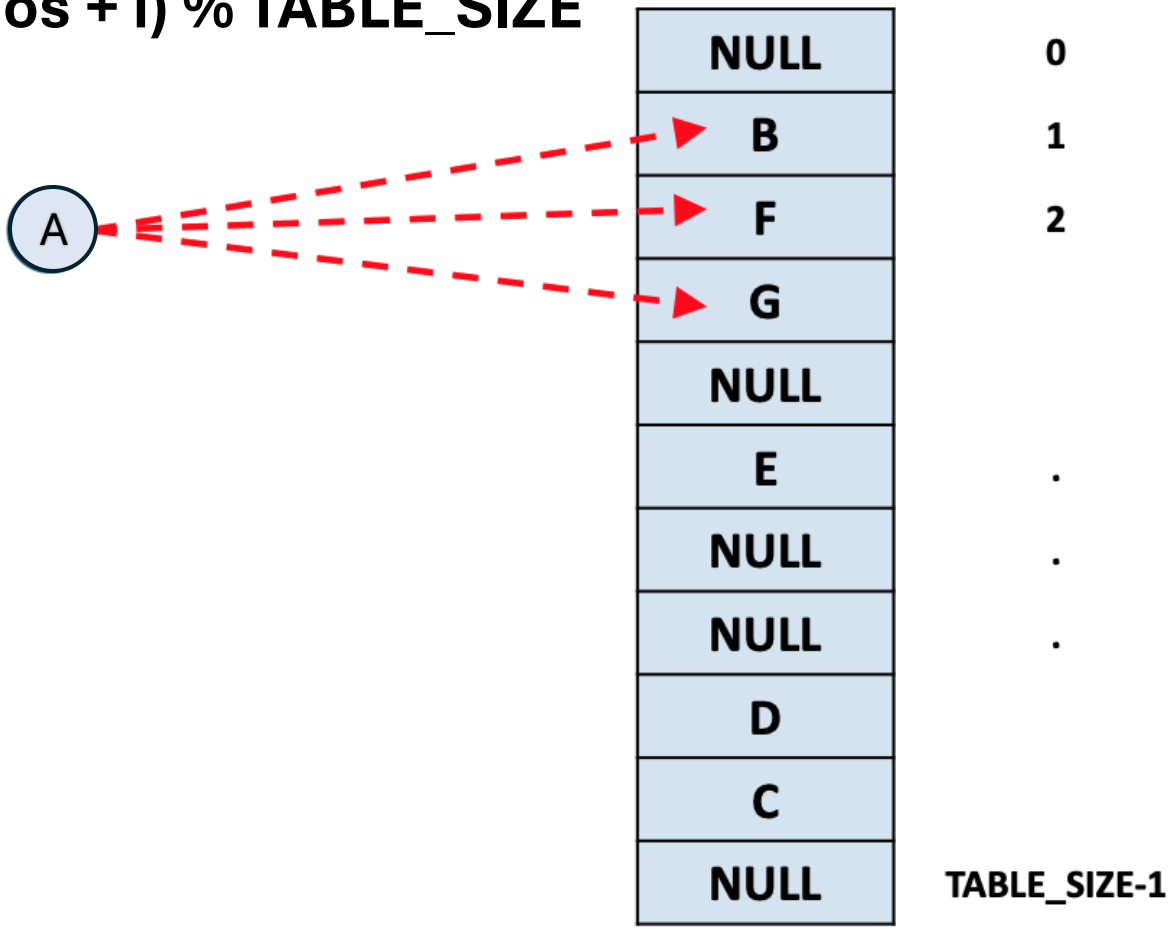
pos = (pos + i) % TABLE_SIZE

Inserção

Quando ocorre uma colisão:

- Calcula a posição inicial:
  - pos = funcao_hash(chave)
- Verifica se a posição está livre:
  - Se **vazia** → insere.
- Se **ocupada** → tenta a próxima.
  - Calcula nova posição:

    pos = (pos + 1) % TABLE_SIZE



Endereçamento aberto: Sondagem linear

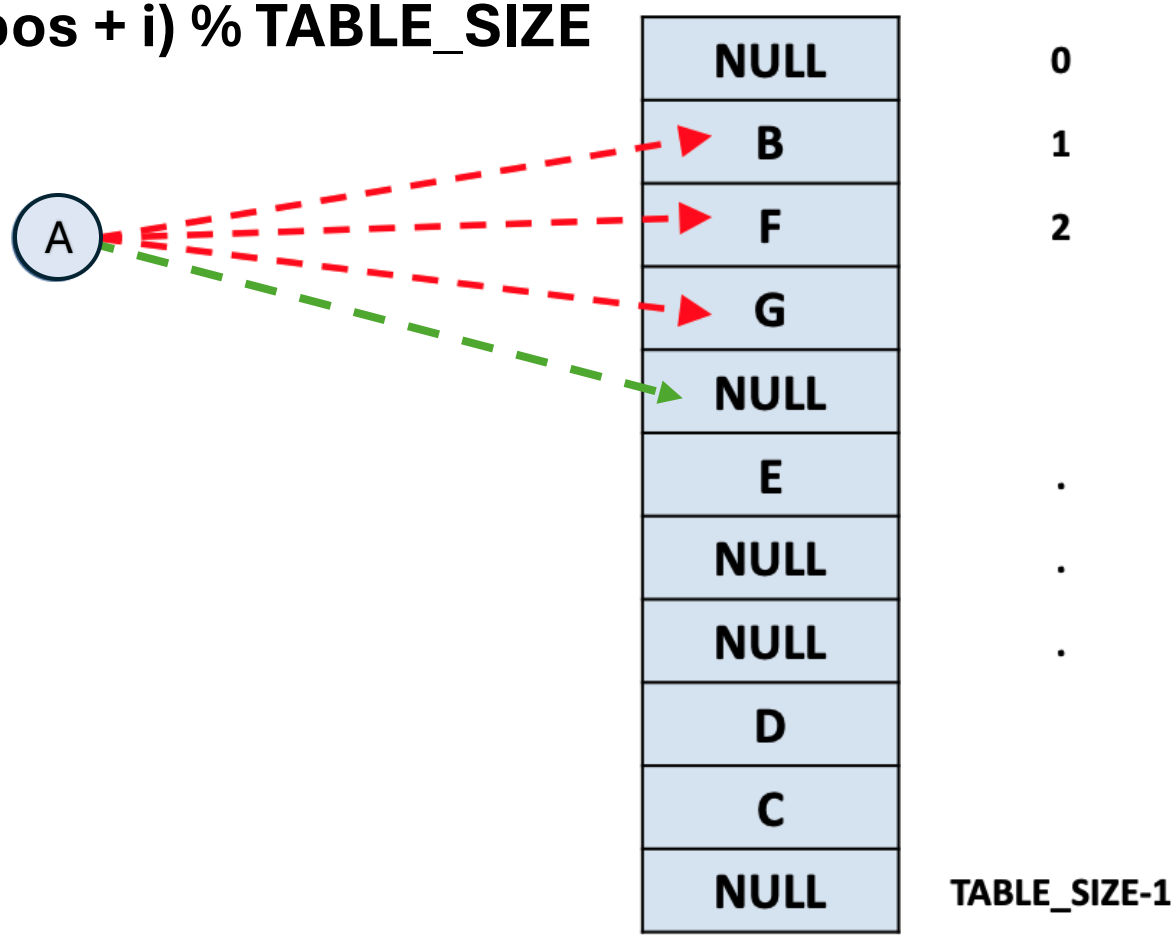
pos = (pos + i) % TABLE_SIZE

Inserção

Quando ocorre uma colisão:

- Calcula a posição inicial:
  - pos = hash(chave)
- Verifica se a posição está livre:
  - Se **vazia** → insere.
- Se **ocupada** → tenta a próxima.
  - Calcula nova posição:

    pos = (pos + 1) % TABLE_SIZE
- Repete até encontrar uma posição livre **ou voltar à posição inicial**

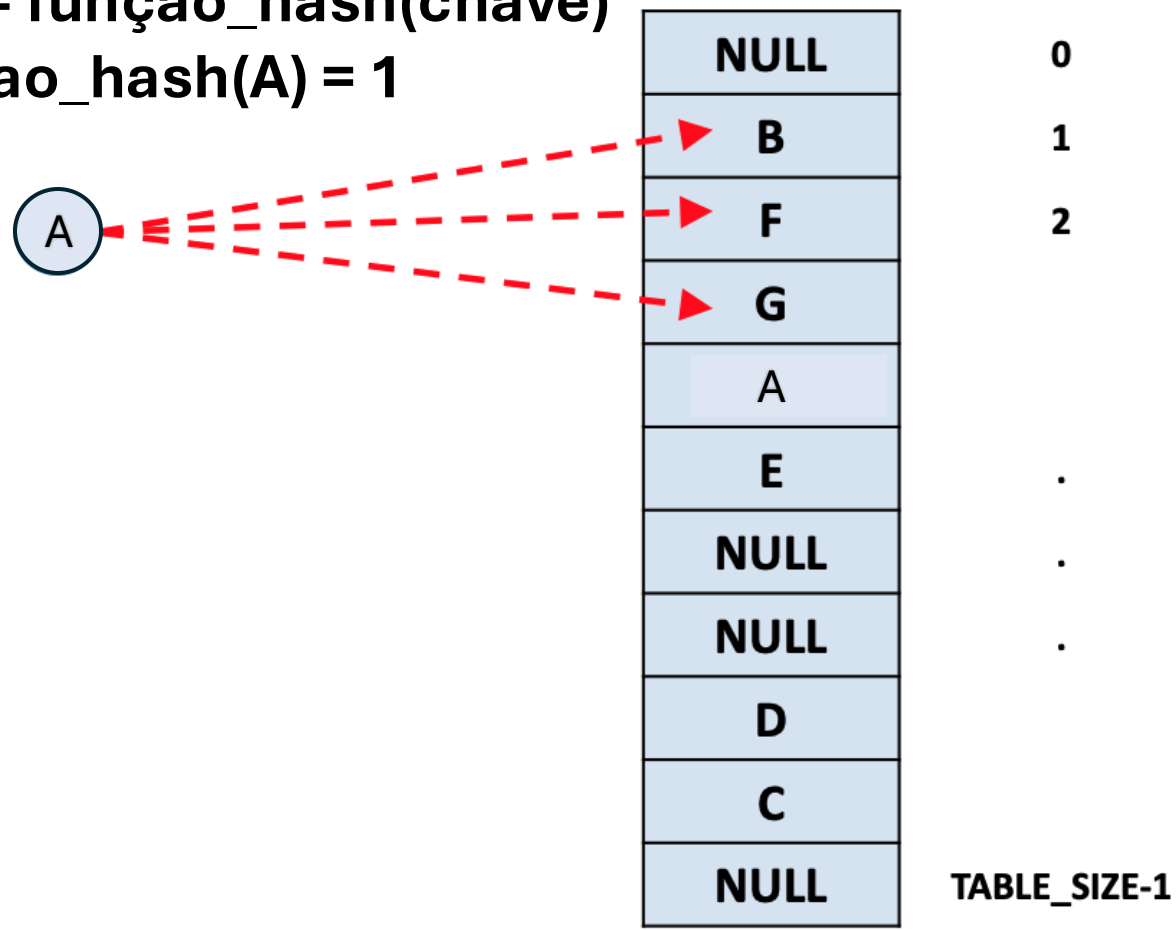


Endereçamento aberto: Sondagem linear

Busca

- Calcula a posição inicial:
  - pos = hash(chave)
- Verifica o conteúdo da posição:
  - Se **vazia (NULL): elemento não existe.**
- Se **a chave é igual: elemento encontrado.**
- Se **a chave é diferente:** tenta a próxima.
- Calcula nova posição:
  - pos = (pos + 1) % TABLE_SIZE

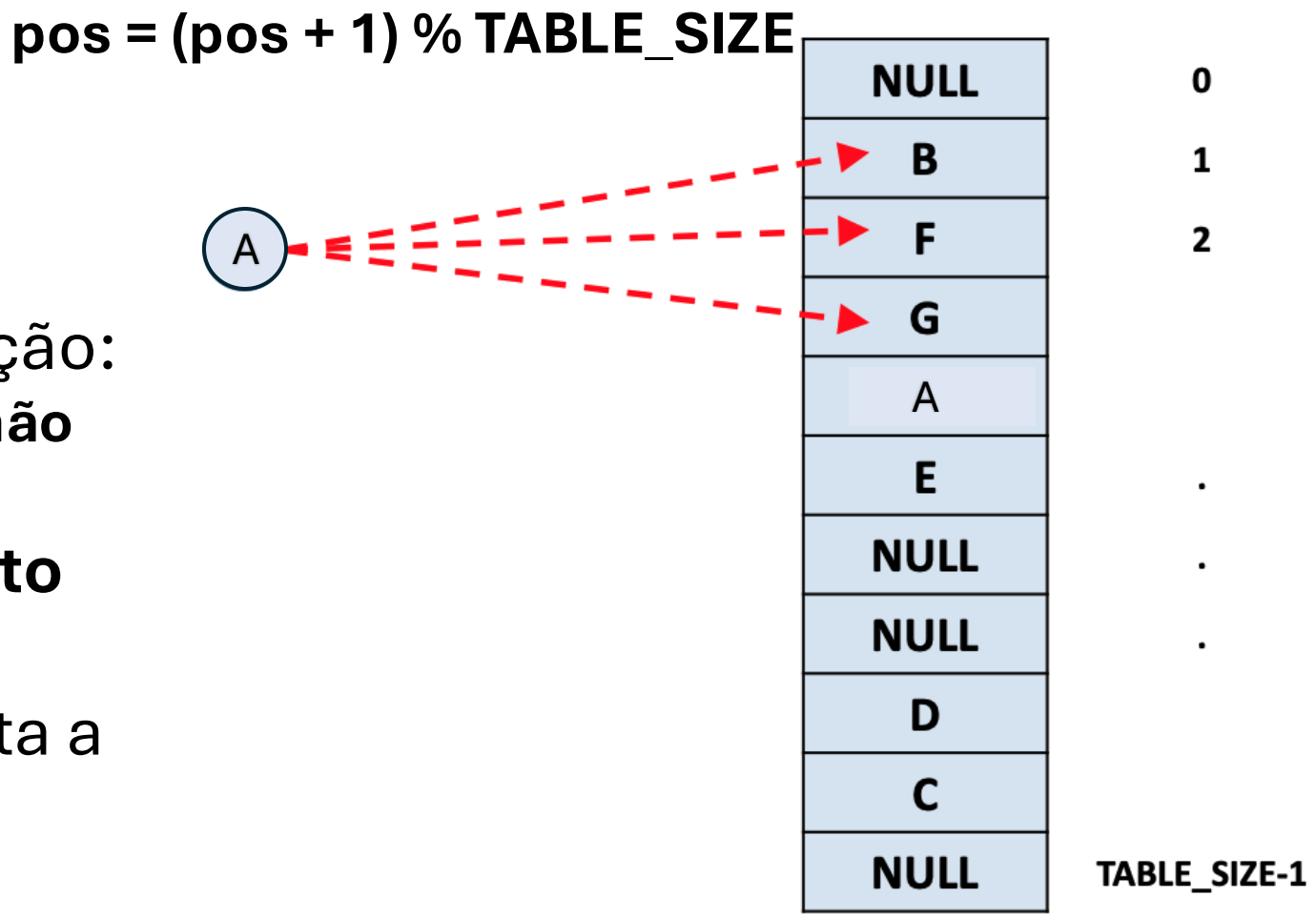
pos = função_hash(chave)
funcao_hash(A) = 1



Endereçamento aberto: Sondagem linear

Busca

- Calcula a posição inicial:
  - pos = hash(chave)
- Verifica o conteúdo da posição:
  - Se **vazia (NULL): elemento não existe.**
- Se **a chave é igual: elemento encontrado.**
- Se **a chave é diferente:** tenta a próxima.
- Calcula nova posição:
  - pos = (pos + 1) % TABLE_SIZE

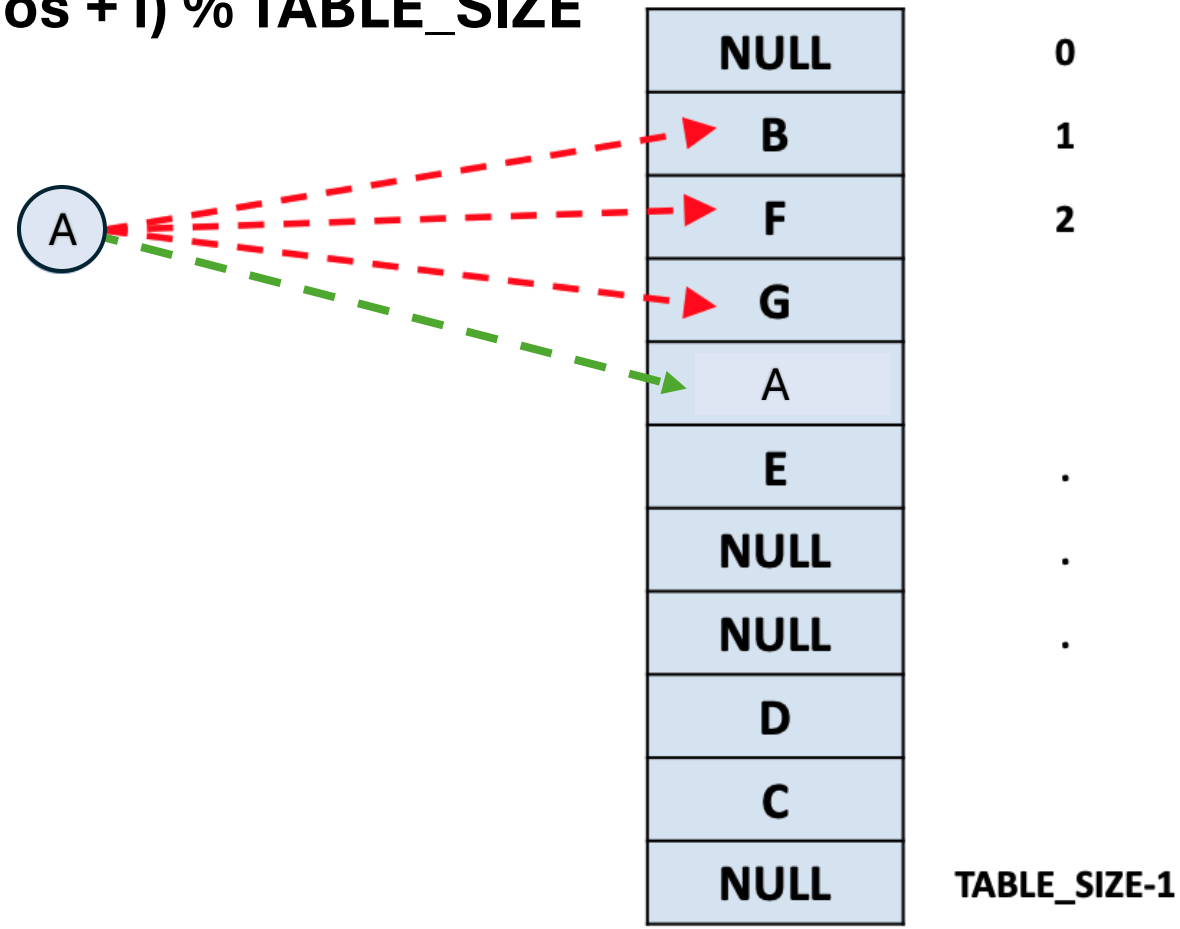


Endereçamento aberto: Sondagem linear

pos = (pos + i) % TABLE_SIZE

Busca

- Calcula a posição inicial:
  - pos = hash(chave)
- Verifica o conteúdo da posição:
  - Se **vazia (NULL): elemento não existe.**
- Se **a chave é igual: elemento encontrado.**
- Se **a chave é diferente:** tenta a próxima.
- Calcula nova posição:
  - pos = (pos + 1) % TABLE_SIZE
- Repete até encontrar o elemento, uma posição vazia ou voltar à posição inicial.



Endereçamento aberto: Sondagem linear

Remoção

NULL	0
B	1
F	2
G	
A	
E	.
NULL	.
NULL	.
D	
C	
NULL	TABLE_SIZE-1

Endereçamento aberto: Sondagem linear

Remoção

Exemplo:

- Se removermos **G** e colocarmos **NULL**

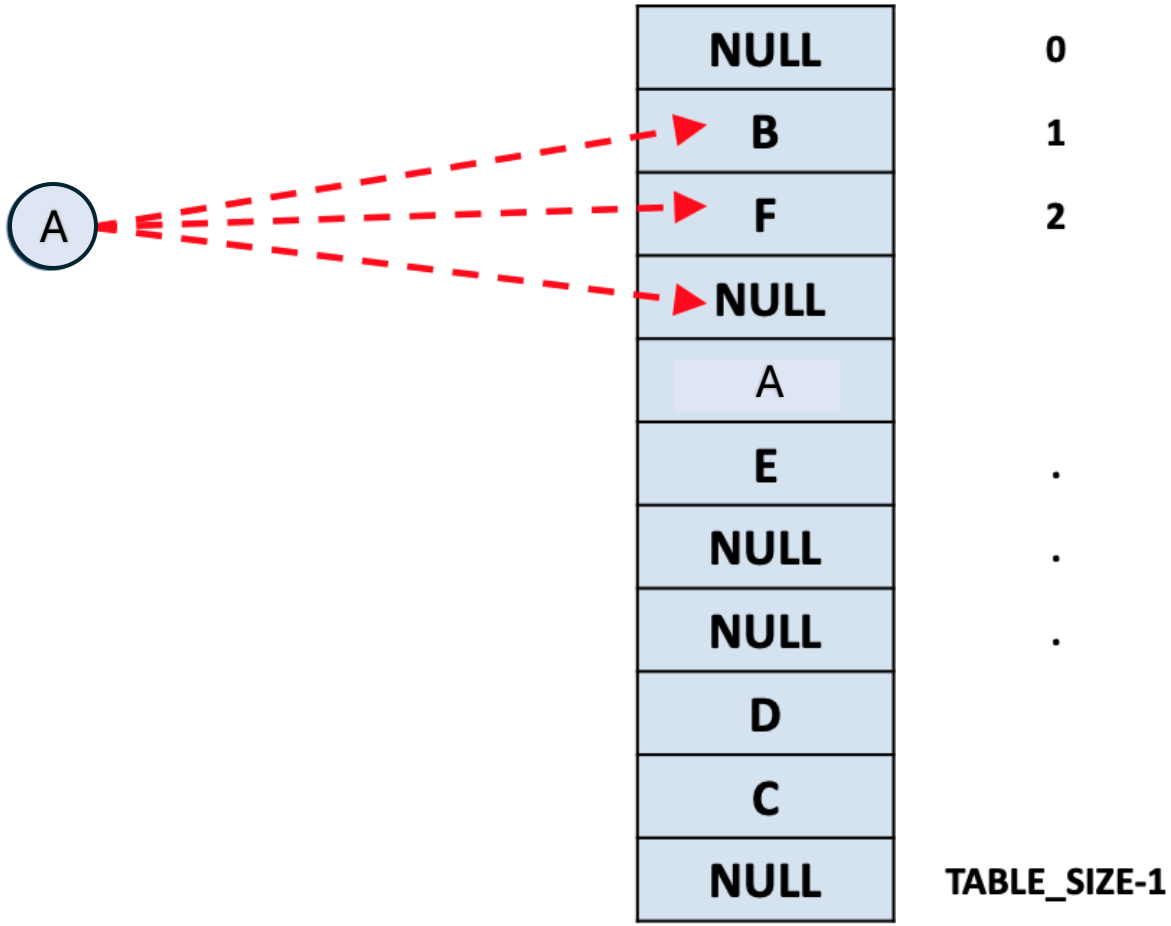
NULL	0
B	1
F	2
NULL	
A	
E	.
NULL	.
NULL	.
D	
C	
NULL	TABLE_SIZE-1

Endereçamento aberto: Sondagem linear

Remoção

Exemplo:

- Se removermos **G** e colocarmos **NULL**
- Não vamos conseguir achar **A**

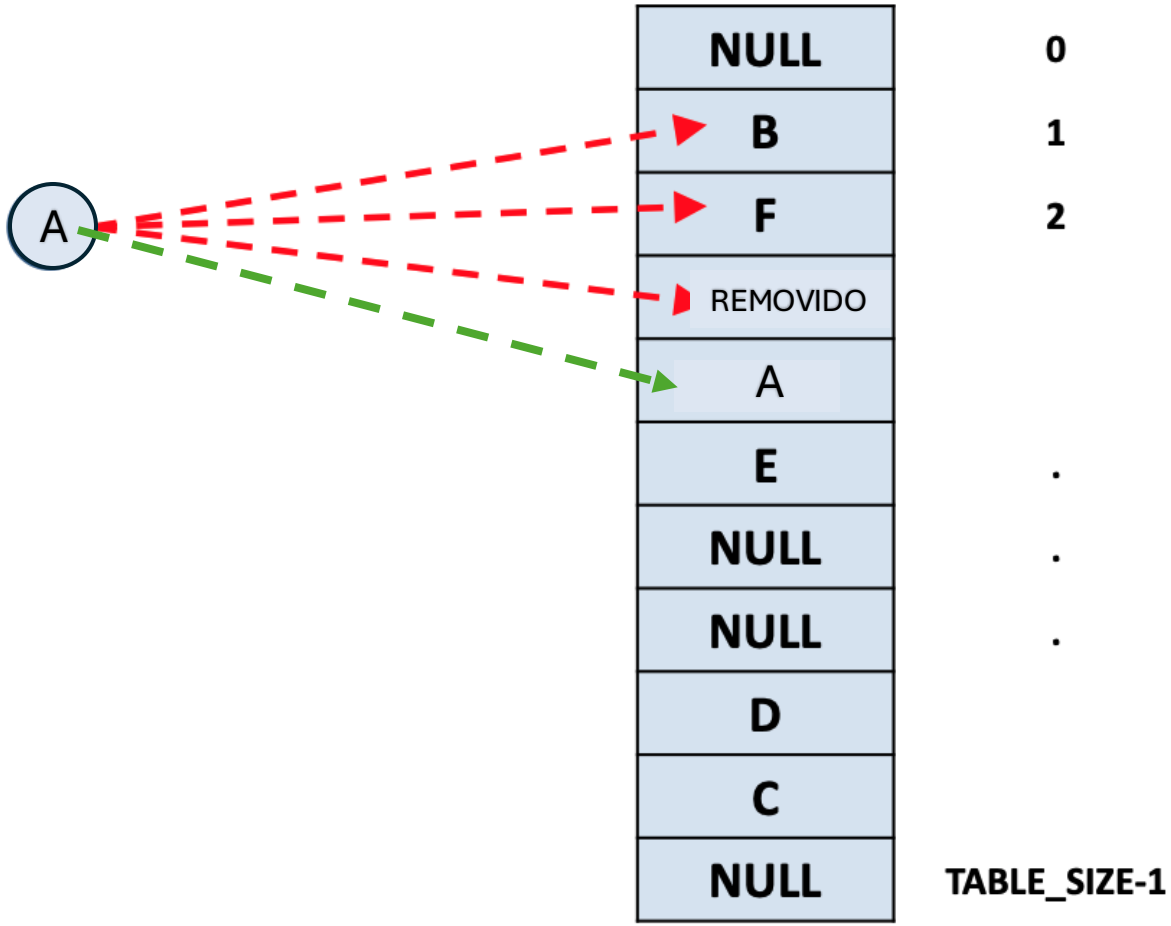


Endereçamento aberto: Sondagem linear

Remoção

Exemplo:

- Se removermos **G** e colocarmos **NULL**
- Não vamos conseguir achar **A**
- **Solução:**
  - Utiliza-se uma **marcação especial** (ex: REMOVIDO)
  - Essa marcação indica que **havia um elemento**, mas ele foi removido.
  - Assim, as próximas buscas continuam sondando normalmente.



Endereçamento aberto: Sondagem linear

NULL	0
NULL	1
NULL	2
NULL	3
NULL	4
NULL	5
NULL	6
NULL	7
NULL	8
NULL	9
NULL	10

CHAVE	POSIÇÃO	INSERÇÃO
A	2	Posição 2 vazia. Insere elemento
B	6	Posição 6 vazia. Insere elemento
C	2	Posição 2 ocupada, procura na próxima posição: 3 Posição 3 vazia. Insere elemento
D	10	Posição 10 vazia. Insere elemento
E	10	Posição 10 ocupada, procura na próxima posição. Como a posição 10 é a última, volta para o início: 0 Posição 0 vazia. Insere elemento

E	0
NULL	1
A	2
C	3
NULL	4
NULL	5
B	6
NULL	7
NULL	8
NULL	9
D	10

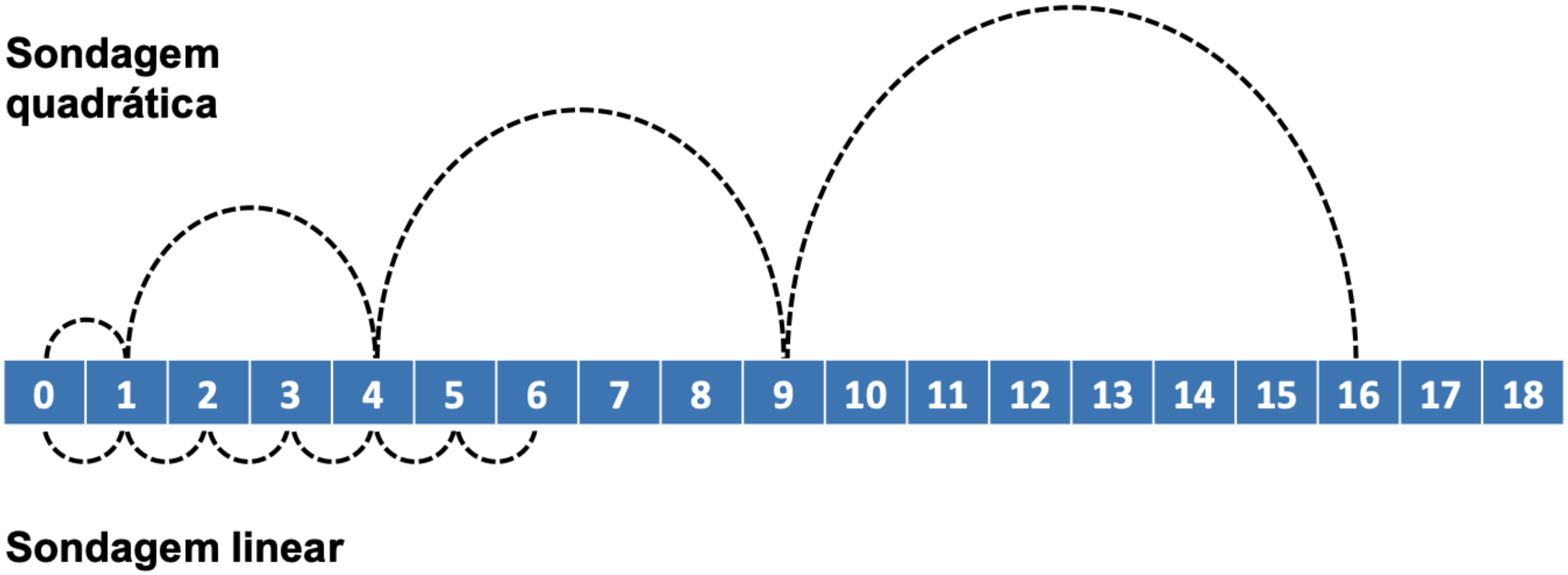
Endereçamento aberto: Sondagem quadrática

- Usa um deslocamento quadrático em relação à tentativa atual.

```
int sondagemQuadratica(int pos, int i, int TABLE_SIZE){
    pos = pos + 2*i + 5*i*i;
    return pos % TABLE_SIZE;
}
```

- Reduz o agrupamento primário mas pode causar **agrupamento secundário**
  - Todas as chaves que produzam a mesma posição inicial também produzem as mesmas posições na sondagem quadrática
- Felizmente, a degradação produzida pelos agrupamentos secundários ainda é menor que a produzida pelos agrupamentos primários

Endereçamento aberto: Sondagem quadrática



Endereçamento aberto: Duplo Hash

- Também conhecida como espalhamento duplo
- Tenta espalhar os elementos utilizando duas funções de hashing:
  - a **primeira** função de hashing, **H1**, é utilizada para calcular a posição inicial do elemento
  - a **segunda** função de hashing, **H2**, é utilizada para calcular os deslocamentos em relação a posição inicial (no caso de uma colisão)

Endereçamento aberto: Duplo Hash

- A posição de um novo elemento na tabela hash é obtida como sendo:
  - **H1 + i * H2**
  - onde **i** é tentativa atual de inserção do elemento
- É necessário que as duas funções de hashing sejam diferentes.
  - A segunda função de hashing não pode resultar em um valor igual a ZERO pois, neste caso, não haveria deslocamento

Endereçamento aberto: Duplo Hash

Funcionamento

- Primeiro elemento (i = 0) é colocado na **posição** obtida por **H1**
- Segundo elemento (colisão) é colocado na posição **H1 + 1 * H2**
- Terceiro elemento (nova colisão) é colocado na posição **H1 + 2 * H2**

```
int duploHash(int H1, int chave, int i, int TABLE_SIZE){
        int H2 = chaveDivisao(chave, TABLE_SIZE-1) + 1;
        return (H1 + i*H2) % TABLE_SIZE;
}
```

Endereçamento aberto: Duplo Hash

Funcionamento

- Utiliza **duas funções de hashing** para calcular a nova posição.

```
int duploHash(int H1, int chave, int i, int TABLE_SIZE){
        int H2 = chaveDivisao(chave, TABLE_SIZE-1) + 1;
        return (H1 + i*H2) % TABLE_SIZE;
}
```

- H1: posição inicial.
- H2: deslocamento entre tentativas.
- Evita agrupamentos e melhora a dispersão.

Encadeamento separado

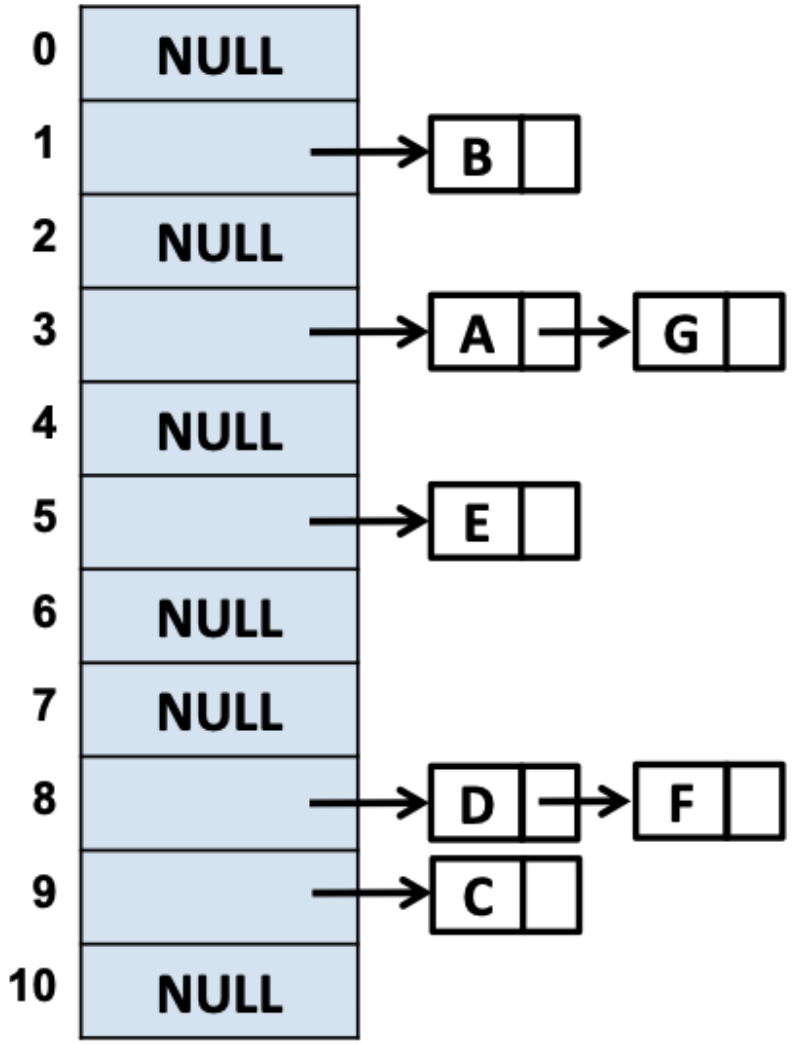
- Também conhecido como *separate chaining*
- Não procura por posições vagas (valor **NULL**) dentro do array que define a tabela
- Armazena dentro de cada posição do array o início de uma lista dinâmica encadeada
  - É dentro dessa lista que serão armazenadas as colisões (elementos com chaves iguais) para aquela posição do array

Encadeamento separado

- Cada índice do array aponta para uma **lista dinâmica**.
- Inserção: adiciona o novo elemento **no início da lista** correspondente.
- Busca: percorre a lista procurando a chave desejada.
- A lista pode ser **ordenada ou não**.

Encadeamento separado

CHAVE	POSIÇÃO
A	3
B	1
C	9
D	8
E	5
F	8
G	3



Encadeamento separado

Características

- A lista dinâmica encadeada mantida em cada posição da tabela pode ser ordenada ou não
- Lista não ordenada
  - Inserção tem complexidade ***O(1)*** no pior caso: basta inserir o elemento no início da lista.
  - Busca tem complexidade ***O(M)*** no pior caso: busca linear
- Desvantagem
  - Quantidade de memória consumida: gastamos mais memória para manter os ponteiros que ligam os diferentes elementos dentro de cada lista

Estrutura de Dados Lineares

Tabela Hash